在 MySQL 中建索引时需要注意哪些事项？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，20031+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

建索引需要关注以下几点：

1）索引不是越多越好。每个索引都占磁盘空间，每次写入都要维护索引的 B+ 树，索引越多写入越慢

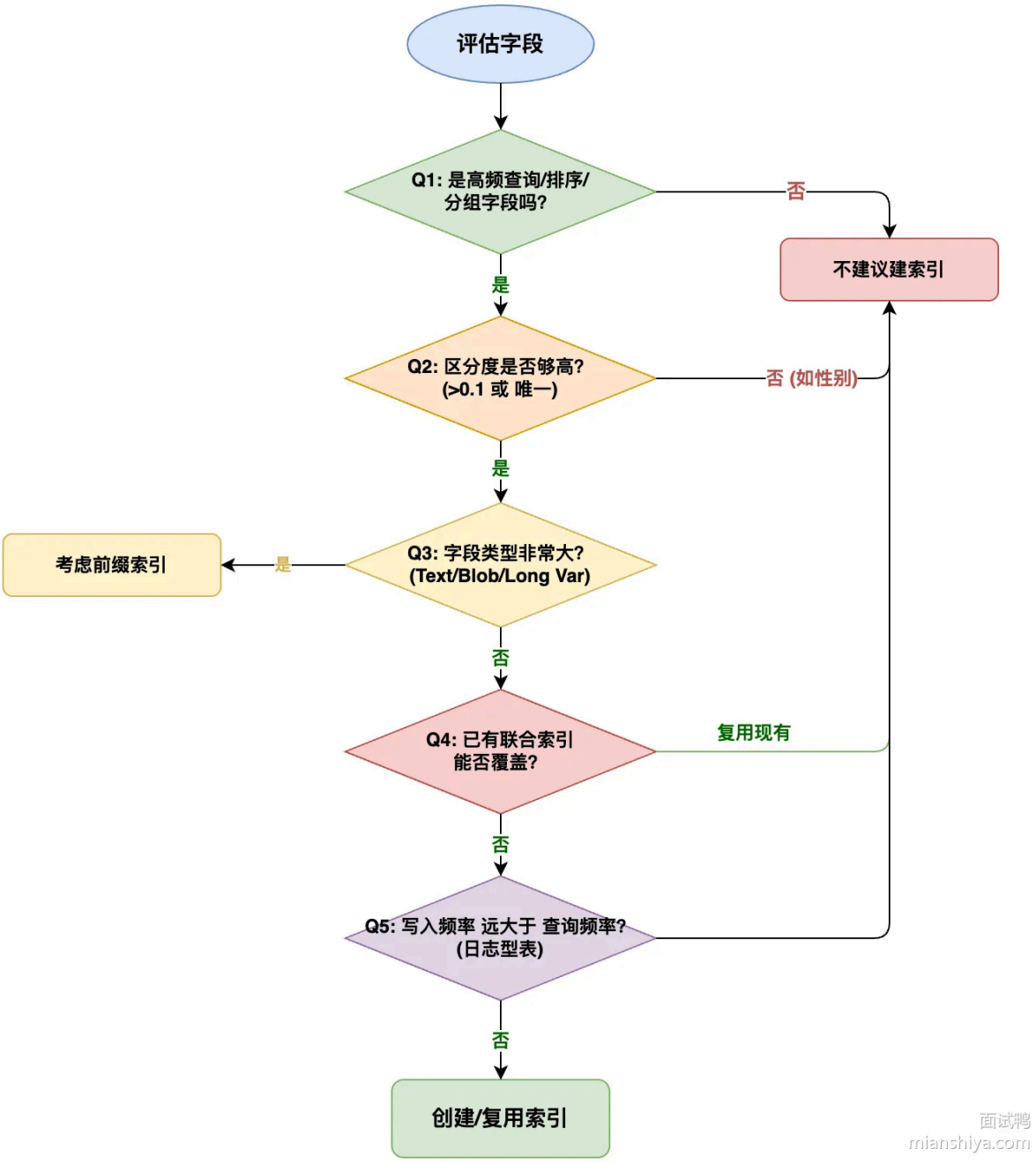
2）区分度太低的字段不要建索引。比如性别字段，就两个值，建了索引也没什么过滤效果。不过也有例外，比如定时任务表大部分是成功状态，只有少量失败，通过失败状态查询能过滤掉 99% 的数据，这时候建索引是有意义的

3）大字段不要建索引。text、longtext 这种字段，索引占用空间大，加载到内存耗时，还可能把其他热点数据挤出 Buffer Pool，得不偿失

4）写多读少的表要慎重。如果修改频率远大于查询频率，索引带来的写入开销可能超过查询收益

5）频繁查询的字段要建索引。where 条件里经常出现的字段，多个字段经常一起查可以考虑联合索引，一个联合索引能覆盖多种查询场景

6）排序、分组字段要建索引。order by、group by、distinct 后面的字段建索引，能避免临时表和文件排序



扩展知识

联合索引的设计原则

建联合索引时列的顺序很关键：

1）**区分度高的列放前面。**区分度越高过滤效果越好，能更快缩小数据范围

2）**查询频率高的列放前面。**保证高频查询能命中索引

3）**范围查询的列放最后。**联合索引遇到范围查询会停止匹配，把范围列放后面能让更多列参与索引查找

4）**利用覆盖索引。**如果查询只需要几个固定字段，把这些字段都放进联合索引，能避免回表

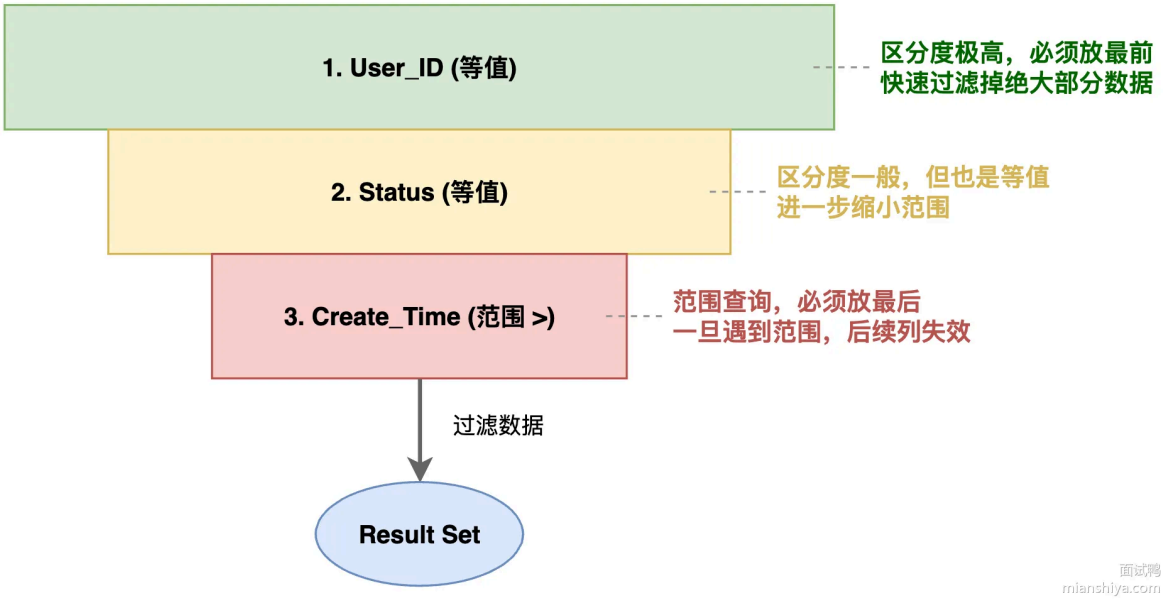
举个例子，电商订单表经常有这几种查询：

```sql
▼sql复制代码-- 查询1：按用户查订单
WHERE user_id = 123

-- 查询2：按用户和状态查订单
WHERE user_id = 123 AND status = 'PAID'

-- 查询3：按用户和时间范围查订单
WHERE user_id = 123 AND create_time > '2026-01-01'
```

建一个联合索引 `(user_id, status, create_time)` 就能覆盖这三种场景。user_id 放最前面因为每个查询都用到，create_time 放最后因为是范围查询。



前缀索引

对于长字符串字段，可以只索引前 N 个字符，叫前缀索引。

```sql
▼sql复制代码-- 对 email 字段的前 10 个字符建索引
ALTER TABLE users ADD INDEX idx_email (email(10));
```

好处是索引小，占用空间少。坏处是可能有重复，区分度下降，而且不能用于 order by 和 group by。

前缀长度怎么选？看区分度。可以用这个 SQL 测试不同长度的区分度：

```sql
▼sql复制代码SELECT
  COUNT(DISTINCT LEFT(email, 5)) / COUNT(*) AS len5,
  COUNT(DISTINCT LEFT(email, 10)) / COUNT(*) AS len10,
  COUNT(DISTINCT LEFT(email, 15)) / COUNT(*) AS len15,
  COUNT(DISTINCT email) / COUNT(*) AS full
FROM users;
```

选一个区分度接近完整字段、长度又不太长的值。一般区分度达到 90% 以上就够用了。

前缀索引长度选择

目标：找到区分度和长度的平衡点

测试方法：计算不同前缀长度的区分度

选择标准：区分度 > 90%，长度尽量短

例如 email 字段：

前5位区分度 60%
ad@email.com
太短

前10位区分度 95%
ad@email.com
合适

前15位区分度 98%
ads@email.e...
没必要

最终选择：email(10)

```
CREATE INDEX idx_email
ON users(email(10))
```

面试鸭

mianshiya.com

索引失效的常见场景

建了索引不一定能用上，以下场景会导致索引失效：

1）对索引列使用函数。`WHERE YEAR(create_time) = 2026` 用不上 create_time 的索引，改成 `WHERE create_time >= '2026-01-01' AND create_time < '2026-01-01'`

2）隐式类型转换。字段是 varchar 但传入数字，MySQL 会做隐式转换，索引失效。比如 phone 是 varchar，`WHERE phone = 13812345678` 会失效，改成 `WHERE phone = '13812345678'`

3）左模糊查询。`WHERE name LIKE '%张'` 用不上索引，只有右模糊 `WHERE name LIKE '张%'` 能用

4）or 连接不同列。`WHERE a = 1 OR b = 2`，如果 a 和 b 分别有索引，可能都用不上。可以改成 UNION

5）联合索引不满足最左匹配。索引 `(a, b, c)`，查询 `WHERE b = 1` 用不上

6）优化器认为全表扫描更快。查询返回数据量超过表的 15%-30%，优化器可能放弃索引

索引监控

建完索引要关注是否被使用。MySQL 5.6+ 可以通过 sys 库查看索引使用情况：

```sql
▼sql复制代码-- 查看未使用的索引
SELECT * FROM sys.schema_unused_indexes;

-- 查看冗余索引
SELECT * FROM sys.schema_redundant_indexes;
```

没用到的索引白白占空间、拖慢写入，应该定期清理。

相关知识点

- 594. MySQL 索引的最左前缀匹配原则是什么？
- 591. MySQL 的索引类型有哪些？

面试官追问

提问：怎么判断一个索引该不该删？

回答：看两个指标。第一是使用频率，通过 `sys.schema_unused_indexes` 或者开启慢查询日志分析，长期没被使用的索引可以考虑删。第二是看是否冗余，比如已经有 `(a, b)` 的联合索引，再单独建一个 `(a)` 的索引就是冗余的，因为联合索引能覆盖单列查询。删索引前最好在低峰期操作，观察一段时间确认没问题。

提问：线上加索引会锁表吗？

> 详细解答可以看：MySQL数据库一亿条数据，怎么快速加索引？

回答：MySQL 5.6+ 支持 Online DDL，大部分情况下加索引不会锁表。InnoDB 会先创建一个临时的新表结构，后台慢慢构建索引，期间 DML 操作的变更会记录到一个日志里，最后应用这些变更。整个过程只在开始和结束时短暂加元数据锁。不过数据量特别大的表，加索引还是会比较慢，建议在业务低峰期操作。

提问：联合索引和多个单列索引有什么区别？

> 详细解答可以看：MySQL 中的覆盖索引和联合索引是什么？索引的最左前缀匹配原则是什么？

回答：联合索引是一棵 B+ 树，多个单列索引是多棵 B+ 树。查询时一般只能走一个索引，联合索引能同时利用多个列过滤，多个单列索引只能选一个走。比如查询 `WHERE a = 1 AND b = 2`，联合索引 `(a, b)` 能同时用 a 和 b 定位，单列索引只能用 a 或 b 其中一个。MySQL 5.0+ 有索引合并优化，某些场景能合并多个单列索引的结果，但效率不如联合索引。